# Experiment - 3

**AIM:** To implement and demonstrate the K-Means clustering algorithm for grouping data samples into clusters based on their similarity.

**Description:** K-Means is an unsupervised machine learning algorithm used for clustering data into K distinct groups. It starts by selecting K initial centroids and assigns each data point to the nearest centroid based on Euclidean distance. The centroids are then recalculated as the mean of the data points belonging to each cluster. This process is repeated until the centroids become stable or the maximum number of iterations is reached. In this implementation, the Iris dataset is divided into 60% training data and 40% testing data. The training data is used to calculate the final centroids, and the testing data is assigned to the nearest cluster.

## Dataset

| sepal.length | sepal.width | petal.length | petal.width | variety |
|---|---|---|---|---|
| 5.1 | 3.5 | 1.4 | 0.2 | Setosa |
| 4.9 | 3 | 1.4 | 0.2 | Setosa |
| 4.7 | 3.2 | 1.3 | 0.2 | Setosa |
| 4.6 | 3.1 | 1.5 | 0.2 | Setosa |
| 5 | 3.6 | 1.4 | 0.2 | Setosa |
| 5.4 | 3.9 | 1.7 | 0.4 | Setosa |
| 4.6 | 3.4 | 1.4 | 0.3 | Setosa |
| 5 | 3.4 | 1.5 | 0.2 | Setosa |
| 4.4 | 2.9 | 1.4 | 0.2 | Setosa |
| 4.9 | 3.1 | 1.5 | 0.1 | Setosa |
| 5.4 | 3.7 | 1.5 | 0.2 | Setosa |
| 4.8 | 3.4 | 1.6 | 0.2 | Setosa |

## Python Code

```
import pandas as pd
import random
data = pd.read_csv("iris.csv", header=None)
data = data.values.tolist()
random.shuffle(data)
X = [row[0:4] for row in data]
train_size = int(0.6 * len(X))
X_train = X[:train_size]
X_test = X[train_size:]
```

```
print("Training data:", len(X_train))
print("Testing data:", len(X_test))
def euclidean_distance(point1, point2):
    distance = 0
    for i in range(len(point1)):
        distance = distance + (float(point1[i]) - float(point2[i])) ** 2
    return distance ** 0.5
def calculate_centroid(cluster):
    centroid = []
    for j in range(len(cluster[0])):
        total = 0
        for i in range(len(cluster)):
            total = total + float(cluster[i][j])
        centroid.append(total / len(cluster))
    return centroid
def kmeans(X, k):
    centroids = random.sample(X, k)
    for iteration in range(100):
        clusters = [[] for i in range(k)]
        for point in X:
            distances = []
            for centroid in centroids:
                distance = euclidean_distance(point, centroid)
                distances.append(distance)
            nearest = distances.index(min(distances))
            clusters[nearest].append(point)
        new_centroids = []
        for cluster in clusters:
            if len(cluster) > 0:
                new_centroids.append(calculate_centroid(cluster))
            else:
                new_centroids.append(random.choice(X))
        if new_centroids == centroids:
            break
        centroids = new_centroids
    return centroids
def predict_clusters(X, centroids):
    predictions = []
    for point in X:
        distances = []
        for centroid in centroids:
            distance = euclidean_distance(point, centroid)
            distances.append(distance)
        cluster = distances.index(min(distances))
        predictions.append(cluster)
    return predictions
```

```
k = 3
centroids = kmeans(X_train, k)
print("\nFinal Centroids:")
for i in range(len(centroids)):
    print("Cluster", i + 1, ":", centroids[i])
train_clusters = predict_clusters(X_train, centroids)
test_clusters = predict_clusters(X_test, centroids)
print("\nTraining Cluster Assignments:")
print(train_clusters)
print("\nTesting Cluster Assignments:")
print(test_clusters)
print("\nNumber of Testing Data Points in Each Cluster:")
for i in range(k):
    count = test_clusters.count(i)
    print("Cluster", i + 1, ":", count)
```

## Output

**Training data:** 90

**Testing data:** 60

**Final Centroids:**

Cluster 1 : [5.046666666666666, 3.4866666666666672, 1.47, 0.25333333333333335]

Cluster 2 : [5.902564102564101, 2.743589743589744, 4.394871794871794, 1.4538461538461536]

Cluster 3 : [6.847619047619048, 3.1238095238095243, 5.723809523809523, 2.1380952380952376]

**Training Cluster Assignments:**

[1, 1, 1, 0, 1, 0, 1, 2, 0, 0, 2, 0, 0, 1, 0, 2, 2, 1, 2, 0, 0, 1, 0, 0, 1, 1, 1, 2, 0, 2, 0, 0, 1, 2, 2, 1, 2, 1, 1, 1, 1, 0, 0, 2, 1, 0, 2, 1, 1, 2, 2, 2, 1, 1, 2, 0, 2, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 2, 1, 1, 1, 2, 2, 2]

**Testing Cluster Assignments:**

[0, 1, 0, 2, 1, 1, 2, 1, 1, 2, 2, 0, 0, 1, 0, 0, 1, 1, 2, 1, 0, 1, 1, 2, 0, 1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 2, 1, 0, 0, 0, 2, 0, 1, 2, 1,

```
0, 0, 2, 0, 2, 1, 0, 2, 0, 2, 0, 0, 0, 1, 2]
```

**Number of Testing Data Points in Each Cluster:**

Cluster 1 : 20

Cluster 2 : 23

Cluster 3 : 17